



MS Thesis – II
Final Semester Progress Report

Code Validation through Machine Learning. A Left Shift focus Strategy

Name	Syed Jehanzeb
ERP	04879
Supervisor	Dr. Faisal Iradat
Co Supervisor	Dr. Belgin Metin
Co Supervisor	Dr. Nazım Taşkın
Date	27th December 2024

Contents

1	Introduction	1
2	RESEARCH QUESTIONS	4
3	Related Work	5
3.1	Traditional Approaches	5
3.2	Machine Learning Approaches	6
3.3	Deep Learning Approaches	7
3.4	Hybrid Approaches	9
3.5	Challenges and Future Directions	9
4	Methodology	11
4.1	Dataset creation and transformation	11
4.2	Model Selection and Training	12
4.3	Explainable artificial intelligence (XAI)	13
5	Analysis	14
5.1	Dataset	14
5.2	Model Selection and Architecture	19
6	Training Process	23
6.1	Data Splitting	23
6.2	Feature Representation	23
6.3	Model Training and Hyperparameter Tuning	24
6.4	Evaluation Metrics	25
7	Model Evaluation and Findings	27
7.1	Logistic Regression	27
7.2	Random Forest Classifier	28

7.3	Support Vector Machine (SVM)	29
7.4	Gradient Boosting Classifier	30
7.5	Extra Trees Classifier	31
7.6	XGBoost Classifier	32
7.7	FLAML AutoML	33
7.8	Summary of Findings	34
8	Explainable artificial intelligence (XAI)	36
8.1	Motivation for XAI	36
8.2	SHAP for Feature Importance Analysis	37
8.3	Impact of XAI on Model Optimization	38
8.4	Discussion and Conclusion	39
9	Discussion	40
9.1	Model Performance Evaluation	40
9.1.1	Comparing Findings with existing literature	41
9.2	Impact of Explainable AI	41
10	Limitations and Future Work	42
11	Conclusion	43

Abstract

This study explores the integration of Explainable Artificial Intelligence (XAI) techniques into machine learning models for API vulnerability detection, focusing on enhancing interpretability and stakeholder trust. Utilizing the Extra Trees Classifier, identified as the most effective model, SHapley Additive exPlanations (SHAP) was employed to provide transparent feature importance analysis. Key findings reveal critical features, such as "authorization" and "token," which significantly influence predictions, aligning with domain-specific security requirements. While XAI techniques improve model transparency. The study underscores the importance of combining high-performing ensemble models with XAI methods to build actionable and trustworthy AI solutions for security applications. Future work should address dataset limitations, scalability, and the development of hybrid models to further enhance performance and interpretability.

Keywords: Explainable Artificial Intelligence (XAI), API Vulnerability Detection, Machine Learning, SHapley Additive exPlanations (SHAP), Feature Importance Analysis, Model Interpretability, Security, Shift-Left Security

Chapter 1

Introduction

As organizations continue to modernize their app portfolios, the number of APIs (Application Programming Interface) in use is projected to exceed one billion by 2031 [10]. Although API allows for a certain level of control, unmanaged APIs introduce significant security risks, including but not limited to authentication, authorization, input validation, data exposure and other security misconfigurations, that cannot be ignored. According to Gartner, by 2025, 50% of all APIs are expected to be unmanaged [14]. These unmanaged APIs can be exploited by attackers to gain unauthorized access, leading to data breaches, service disruptions, and potential compliance violations. Organizations must address these risks by implementing robust API management and security strategies to safeguard their systems.

The Open Web Application Security Project (OWASP) API Security Top Ten [13] is an industry recognized guideline for some of the most critical API vulnerabilities. The Broken Authentication and Broken Object Level Authorization are the top two critical vulnerabilities on the list. The list contains an in depth analysis, explanation, scenarios and preventative measure for each of the top 10 vulnerabilities. This standardization of Vulnerability data has laid down a blueprint for other developers and organizations working to safeguard their data. OWASP which was founded in 2001 [1] to improve security in the software arena realized the importance of API security and started working on API security in 2019 [13].

Shift Left vs Shift Right Security

Shift-left security [15] involves integrating security measures early in the software development lifecycle, focusing on identifying and addressing vulnerabilities during the design, development, and testing stages. This proac-

Shift Right vs Shift Left
Summary of Papers Classified during Thesis Part I

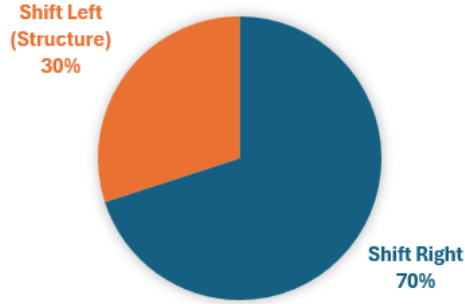


Figure 1.1: Shift Right vs Shift Left

tive approach ensures that security issues, such as broken authentication, insecure data exposure, and improper input validation, are mitigated before products are deployed. By incorporating techniques like static code analysis [49], security testing in CI/CD pipelines, and secure API design principles, shift-left security reduces the attack surface, lowers costs associated with post-production fixes, and ensures more robust APIs are deployed.

Shift-right security [41] focuses on monitoring and responding to security threats after APIs are deployed in production [45]. It emphasizes real-time detection, logging, and incident response, ensuring that vulnerabilities or attacks, such as zero-day exploits or runtime misconfigurations, are identified and addressed quickly. Unlike shift-left, which prevents issues during development, shift-right assumes that security incidents will occur and focuses on minimizing damage through continuous monitoring [41], user behavior analysis, and rapid recovery mechanisms. This approach complements shift-left by providing ongoing protection and adaptability in dynamic, real-world environments.

In Part I of this thesis, a review was conducted to evaluate the current direction of research in API security. The findings highlighted a predominant focus on shift-right security [3, 8, 9, 26, 33, 38, 42, 44], where 70% of the papers were classified as Shift Right and only 30% was classified as Shift Left [2, 11, 12, 16, 18, 25, 48], all of shift left security papers have sub classification of structure or framework related. There was little emphasis on improving the quality of coding in terms of API security i.e. Static Testing [43]. Though

there is some work on security of software application [6, 47]. The reason can be appropriated to programming language dependency and the variety of coding style that limits the applicability of the research output.

Conclusively, the security arena is focused more on testing the security of finished product i.e. Shift Right Security rather than building security within the production cycle i.e. Shift Left Security. This thesis takes advantage of this gap and proposes a solution or a tool which would identify the security gaps during the coding of the API.

This thesis works to introduce static testing for API development through leveraging **Machine Learning (ML)** models to detect vulnerabilities in API authentication code. The OWASP API Top Ten [13] vulnerabilities will serve as a blueprint for Machine Learning algorithms. To complement this, we also incorporate **Explainable AI (XAI)** techniques to provide transparency and interpretability in the decision-making process of the models. XAI techniques are essential for making AI-driven security solutions more accessible and trustworthy by enabling developers to understand and explain the reasoning behind vulnerability predictions made by the models.

Ultimately, this work seeks to explore the efficacy of Machine Learning models for AI-powered security solutions, complying with OWASP guidelines, which not only improve vulnerability detection but also ensure that the underlying models are interpretable and actionable by developers, promoting the widespread adoption of AI-driven Shift Left Security practices.

Chapter 2

RESEARCH QUESTIONS

This study aims to identify the possibility of developing a model to judge code security.

RQ1 How does Machine Learning help identify the code security level?

RQ2 How can Explainable AI (XAI) identify the key points influencing the output of Machine Learning Models ?

The second question will explain the reason for how machine learning is working, what are the key areas of concern, and if the model is true.

Chapter 3

Related Work

Building security or quality within the production cycle isn't new. This has been in practice since the 1980s in Japan and was standardized by W. Edwards Deming in the famous 1987 paper. He emphasized in his quality management philosophy that quality be built into the product itself [29]. Security is an important part of the quality of a product [5]. The age-old practice of testing the quality of a product after it has been developed is costly and inefficient. If security is built into the product in the first place, it saves a lot of testing and revision time, eventually speeding up the product development lifecycle [43].

3.1 Traditional Approaches

Traditional methods for software vulnerability detection have primarily relied on static and dynamic analysis tools. Static analysis tools, such as *Flawfinder*, *RATS*, and *CPPCHECK*, have been effective in identifying vulnerabilities in programming languages like C/C++ and Java. However, these tools often suffer from high false positive rates [6,47]. Dynamic analysis tools, such as *Sydr-Fuzz*, are useful in post-deployment phases but are less efficient for early-stage detection [45].

Despite their utility, these traditional tools lack the ability to generalize across diverse codebases and often require significant manual effort to address detected issues. These limitations have paved the way for more advanced methods based on machine learning and deep learning [46].

Aspect	Static Analysis Tools	Dynamic Analysis Tools
Key Points	Analyzes source code without executing it	Analyzes program behavior during execution
Benefits	• Detects potential issues early • No need to execute the program • Comprehensive coverage of code paths	• Identifies runtime issues like memory leaks • Real-world input testing • Better insights into program behavior
Challenges	• May generate false positives • Limited to code-level insights • Cannot detect runtime errors	• Requires execution environment setup • Performance overhead during analysis • May miss dormant code paths

Table 3.1: Comparison of Static and Dynamic Analysis Tools

3.2 Machine Learning Approaches

Machine learning has emerged as a promising alternative to traditional methods. Machine Learning utilizes data-driven techniques for identifying patterns and anomalies in code that may indicate vulnerabilities. Machine learning algorithms are capable of learning from and making decisions or predictions based on data. ML algorithms build models from sample data, enabling systems to improve performance on specific tasks without being explicitly programmed. In the context of security analysis, ML offers tools to detect patterns, anomalies, or vulnerabilities in code, assisting developers in identifying and mitigating potential issues in systems.

Several Machine Learning techniques have been applied to this field, with varying degrees of success. Supervised learning approaches are often used, where labeled datasets of vulnerable and non-vulnerable code are employed to train classifiers. Techniques like logistic regression, support vector machines (SVM), and decision trees are among the traditional approaches [21]. As the

solution closely resembles an NLP (Natural Language Processing), suitable [19] machine learning technique can be adopted.

Algorithm	Brief Description	Strengths	Weaknesses	Best Use Cases
Logistic Regression	Predicts probabilities for binary classification using a sigmoid function.	Simple, interpretable, provides probabilities.	Assumes linear relationships, struggles with complex patterns.	Binary classification tasks, medical diagnoses.
Support Vector Machines (SVM)	Finds an optimal hyperplane to separate classes in feature space.	Effective for small, high-dimensional datasets.	Computationally expensive, sensitive to kernel selection.	Text/image classification, small datasets.
Decision Trees	Creates a tree by iteratively splitting data based on feature values.	Simple, interpretable, handles mixed data types.	Overfitting, unstable (small data changes affect results).	Quick prototyping, interpretable decision-making.
XGBoost	A boosting algorithm that builds sequential trees to optimize errors.	High accuracy, efficient, robust.	Complex to tune, prone to overfitting without care.	Competitive machine learning tasks, structured data.
Extra Trees	Randomized decision tree splits increase generalization.	Fast training, reduces overfitting.	Random splits may yield less optimal models.	Large datasets, feature selection.

Table 3.2: Summary of Machine Learning Algorithms with Brief Descriptions

Marjanov et al. [24] discuss ML pipelines for vulnerability detection, emphasizing the importance of high-quality datasets and the challenges of overfitting and generalization. Token-based sequence processing and statistical feature extraction are commonly used in ML-based approaches.

Integrating AI into vulnerability detection has shown promising results. Rajapaksha et al. [34] proposed a machine learning-based method for detecting vulnerabilities in C/C++ applications. Their approach achieved 85% in multi-class classification indicating the efficacy of machine learning models in vulnerability detection in source code. Additionally, they applied Explainable AI techniques to assist developers in identifying and understanding vulnerable code segments. The models were trained on datasets containing actual source code, ensuring high generalizability.

3.3 Deep Learning Approaches

Deep Learning (DL) is a subset of machine learning that uses neural networks with multiple layers to automatically learn representations from data. Deep Learning models can extract high-level, abstract features from raw input data, unlike traditional machine learning models, that often require extensive feature engineering. This capability makes Deep Learning particularly

suitable for modeling complex patterns in data, such as code structures and contextual relationships in software.

Deep learning models have gained traction due to their ability to model complex patterns.

- **Sequence-Based Models:** Sequence-based models treat source code as a series of tokens or instructions and analyze temporal or contextual relationships between them. By leveraging architectures such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and transformers, these models excel at understanding the sequential nature of code execution and dependencies. Models such as *VulDeePecker* and *SySeVR* leverage RNNs and transformers to process code sequences, demonstrating strong performance in contextual vulnerability detection [22, 23].
- **Graph-Based Models:** Graph-based models represent source code as graphs, such as Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), or Program Dependency Graphs (PDGs), capture the structural and syntactic relationships in the code. Graph Neural Networks (GNNs) are then used to process these graph representations and extract meaningful embeddings. For instance, *GraphcodeBERT* combines graph-based embeddings with pre-trained transformer models to capture both semantic and structural aspects of code, while *VulLMGNN* [23] integrates graph embeddings with language models to achieve excellent performance in vulnerability detection. These models are useful for identifying complex vulnerabilities related to data flows, control flows, or multi-function interactions within large codebases.

Russell et al. [37] developed a large-scale function-level vulnerability detection system using convolutional and recurrent neural networks. By standardizing source code representations with a custom lexer, the authors minimized noise and improved feature extraction. The CNN+RF hybrid model achieved superior performance compared to traditional static analysis tools like Clang and Flawfinder. This study highlights the potential of combining deep learning with ensemble methods for precise and scalable vulnerability detection.

3.4 Hybrid Approaches

Hybrid methods that combine token-based and graph-based techniques are gaining attention. For example, *Vul-LMGNN* combines graph neural networks with pre-trained code language models, effectively utilizing both structural and contextual features [23]. Similarly, *EnStack* employs ensemble techniques with pre-trained language models like *CodeBERT* to enhance vulnerability detection [39].

Method	Technique	Highlights
Vul-LMGNN	Combines graph neural networks with pre-trained code language models	Utilizes both structural and contextual features for enhanced vulnerability detection.
EnStack	Employs ensemble techniques with pre-trained language models like CodeBERT	Enhances vulnerability detection by leveraging pre-trained language model capabilities.

Table 3.3: Hybrid Methods Combining Token-Based and Graph-Based Techniques

Deep learning has shown promise in enhancing vulnerability detection through feature representation learning. Russell et al. [37] demonstrated that combining CNNs and random forests improves the granularity and scalability of vulnerability detection systems. Additionally, recent studies have highlighted the potential of large language models (LLMs), such as *GPT-2*, *RoBERTa*, and *CodeT5*, in vulnerability detection [7, 23].

3.5 Challenges and Future Directions

A significant challenge in ML-based vulnerability detection is the lack of diverse and high-quality datasets. The *DiverseVul* dataset offers extensive coverage with over 18,000 vulnerable functions across 150 CWEs, facilitating better model evaluation [7]. However, issues such as labeling noise and imbalanced datasets persist [7].

Existing papers cited above span a wide variety of applications of Artificially Intelligent solutions for Left Shift security, this paper focuses on Python code with vulnerabilities in accordance with OWASP's Top Ten API Security vulnerabilities. This paper explores various models primarily in the Machine Learning domain with different optimization techniques and applies XAI on our best performing model using SHAP. The goal of this paper is to utilize multiple approaches for Machine Learning algorithms on vulnerability detection in source code in accordance with OWASP's Guidelines for vulnerabilities.

Chapter 4

Methodology

The aim of this paper is to explore the ability Machine Learning models in detecting vulnerabilities in source code, case in point, Static Testing. We will do that within the context of OWASP's Top Ten API Security vulnerabilities. This study employs a structured approach to develop a machine learning pipeline for the classification of coding language, focusing on dataset creation, machine learning model selection, its training process, and explainable artificial intelligence (XAI). The methodology is divided into four phases:

1. Dataset creation and transformation
2. Model Selection and Architecture
3. Model Training Process
4. Explainable artificial intelligence (XAI)

4.1 Dataset creation and transformation

As this research could not find any static testing tool for API Security, as such, there was no dataset that can be relied for machine learning. There were two ways to move forward. Either scrap authentication codes from famous libraries or build a new list of codes. As the scrap codes will not cover all areas identified by OWASP guidelines and neither will there be suitable insecure code available, we ended up with building a labeled code library. The code library consists of API authentication source code for each of the OWASP's API Security Top Ten Vulnerabilities [13], with three common

vulnerabilities for each of them and secure and insecure labeled versions for each vulnerability . This came to a dataset with 60 code snippets for authentication, perfectly distributed according to vulnerability category.

$$\begin{array}{ccccccc} 10 & \times & 3 & \times & 2 & = & 60 \\ (OWASP\ API & & (Vulnerabilities & & (Secure/Insecure & & (Code \\ Top\ Ten) & & Types) & & Versions) & & Snippets) \end{array}$$

We then stripped the code of it’s non-essential/bias inducing components, such as comments, followed with code vectorization to make it suitable for machine learning algorithms.

4.2 Model Selection and Training

As the dataset is a labeled dataset with binary results, we used the models to perform supervised machine learning with classification output. Supervised Machine Learning is a branch of artificial intelligence where models are trained using labeled data to make predictions or decisions. Each data point is provided with a label which the model initially tries to predict which in turn is supported or corrected by the model essentially using the label.

As there is a sort of similarity between our research and Natural Language Processing, a field within computer science focusing on the ability of computers to understand languages, we started off with suitable models such as Logistic Regression and Support Vector Machine (SVM) [19]. Furthermore we performed a GridSearch spanning a wide range of model hyperparameters. To conclude we finally utilized Auto ML to explore several machine learning techniques.

As the dataset is limited but well structured, it might perform adequately with classical models. However, the task is somewhat unique and to we cannot definitively pick any model without testing it and even then some. AutoML [17] is one such library that explores several models quickly, the best performing model can then be GridSearch-ed with hyperparameters. This is relatively possible due to our dataset being small.

4.3 Explainable artificial intelligence (XAI)

Our intention was to attempt a justifiable explanation for our results, each step pertaining to the model would ensure that this model can be explained on, or an attempt thereof, by Explainable AI. Our goal was to find an Explainable AI (XAI) model to identify critical or the most contributing factors towards the achieved result. We decided on SHAP (Shapley Additive Explanations), namely its feature importance plot, owing to its clear and understandable presentation, after considering other libraries.

Chapter 5

Analysis

5.1 Dataset

The dataset was created with the help of a security professional specifically for this thesis. The area targeted for the thesis is Authentication source code in Python, which has remained the top three in Owasp’s Top Ten Vulnerabilities list [13] during the past few years. This dataset contains a variety of Python code simulating different vulnerabilities in API authentication code, focusing on authentication-based features such as logging in and signing up. The types and variety of the dataset was inspired from OWASP API Top Ten vulnerability list [13]. The dataset consists of 60 code snippets, each labeled with primary and secondary vulnerability categories. It is compact yet diverse, allowing for a focused analysis on authentication vulnerabilities.

Each entry has a primary label linking it to an OWASP API Security Vulnerability, along with a secondary label highlighting the specificity of the vulnerability, such as `Inadequate user validation`. The dataset contains two entries per secondary label: one with the error and one with the error resolved. This results in a balanced dataset, with each sub-label containing evenly distributed code for secure and insecure practices.

While focused on authentication, this dataset may lay groundwork for future Shift Left Source Code solutions, due to its custom nature and the fact that we were unable to find many research papers that exhaustively used Machine Learning techniques for vulnerability detection in code having vulnerabilities with respect to OWASP’s Top Ten API Security Guidelines, which could be expanded to include other vulnerabilities like authorization

and data validation, broadening its applicability.

In the following example, an API endpoint allows users to access details of any user by specifying the user ID in the URL. The endpoint fails to check whether the requesting user is authorized to access the requested user's details.

```
1 from flask import Flask, request, jsonify
2 app = Flask(__name__)
3 # Mock user data
4 users = {
5     1: {"name": "Alice", "email": "alice@example.com"},
6     2: {"name": "Bob", "email": "bob@example.com"}
7 }
8 @app.route('/user/<int:user_id>', methods=['GET'])
9 def get_user(user_id):
10     user = users.get(user_id)
11     if user:
12         return jsonify(user)
13     else:
14         return jsonify({"error": "User not found"}), 404
15 if __name__ == '__main__':
16     app.run(debug=True)
```

Listing 5.1: Vulnerable Code for Broken Object Level Authorization for Inadequate User Validation Vulnerability

```
1 from flask import Flask, request, jsonify
2 app = Flask(__name__)
3 # Mock user data
4 users = {
5     1: {"name": "Alice", "email": "alice@example.com"},
6     2: {"name": "Bob", "email": "bob@example.com"}
7 }
8 # Mock API keys for users
9 api_keys = {
10     "key_for_alice": 1,
11     "key_for_bob": 2
12 }
13 def get_user_id_from_api_key(api_key):
14     return api_keys.get(api_key)
15 @app.route('/user/<int:user_id>', methods=['GET'])
```

```

16 def get_user(user_id):
17     api_key = request.headers.get('X-API-Key')
18     if not api_key:
19         return jsonify({"error": "Missing API key"}), 401
20     requesting_user_id = get_user_id_from_api_key(api_key)
21     if not requesting_user_id:
22         return jsonify({"error": "Invalid API key"}), 403
23     if requesting_user_id != user_id:
24         return jsonify({"error": "Unauthorized access"}), 403
25     user = users.get(user_id)
26     if user:
27         return jsonify(user)
28     else:
29         return jsonify({"error": "User not found"}), 404
30     if __name__ == '__main__':
31         app.run(debug=True)

```

Listing 5.2: Secure Code for Broken Object Level Authorization for Inadequate User Validation Vulnerability

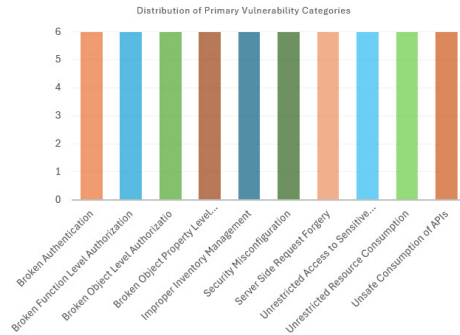


Figure 5.1: Distribution of Primary Vulnerability Categories

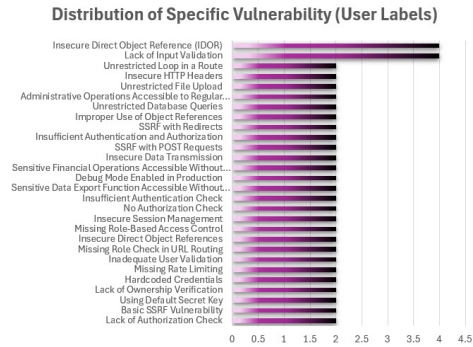


Figure 5.2: Distribution of Specific Vulnerability Types (User Labels)

Code comments were removed as they could indirectly act as indicators for vulnerabilities, thereby biasing the theoretical AI solution for source code discussed later in this paper. Additionally, stopwords were filtered out to reduce complexity in the models.

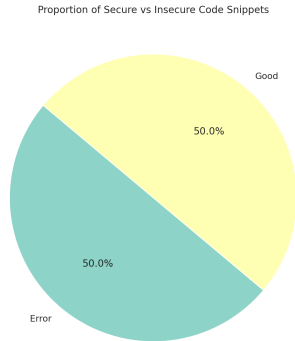


Figure 5.3: Proportion of Secure vs Insecure Code Snippets

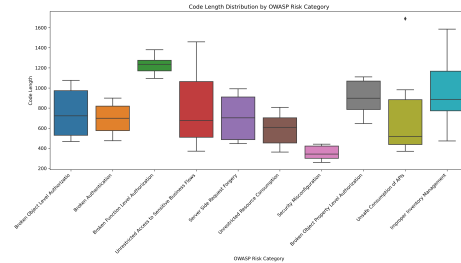


Figure 5.4: Code Length Distribution by OWASP Risk Category

Vectorization and Label Encoding The dataset underwent vectorization and label encoding. Vectorization is a preprocessing step that transforms textual data into numerical representations suitable for machine learning model input. Two vectorization techniques were employed:

1. **Count Vectorization:** This method captures the frequency of each token within a code snippet, resulting in a sparse matrix representation. It provides a direct representation of token occurrence without considering their importance across the dataset.
2. **TF-IDF (Term Frequency-Inverse Document Frequency):** This method assesses the relative importance of tokens by balancing their frequency in individual snippets with their prevalence across the entire dataset. TF-IDF assigns higher weights to tokens that are unique to specific code snippets while down-weighting commonly occurring ones, such as programming keywords.

Both vectorization methods utilized the filtered tokens generated during preprocessing. The transformation steps were as follows:

```
1 # Count Vectorization
2 vectorizer = CountVectorizer()
3 count_vectors = vectorizer.fit_transform(
```

```

4         df['filtered_tokens'].apply(lambda tokens: ' '.join(
5             tokens))
6
7     # TF-IDF Vectorization
8     tfidf_vectorizer = TfidfVectorizer()
9     tfidf_vectors = tfidf_vectorizer.fit_transform(
10         df['filtered_tokens'].apply(lambda tokens: ' '.join(
11             tokens))
12     )

```

Listing 5.3: Vectorization Process

The dataset's **Result** column, which indicates whether a code snippet contains a vulnerability or its resolved counterpart, was numerically encoded for compatibility with machine learning algorithms. The label encoding ensured that models could process the categorical data. A binary mapping was applied: **Error** was encoded as 0, and **Resolved** was encoded as 1. The following Python snippet illustrates the process:

```

1 from sklearn.preprocessing import LabelEncoder
2
3 # Encode the 'Result' column
4 le = LabelEncoder()
5 df['Result_encode'] = le.fit_transform(df['Result'])

```

Listing 5.4: Label Encoding

The vectorization and label encoding steps were essential in preparing the dataset for downstream tasks, ensuring model compatibility and efficient processing for machine learning models.

Stopwords Used for Filtering A custom stopwords list was used to filter out irrelevant tokens, such as common programming keywords and special characters. This helped reduce noise in the dataset, improving the performance of machine learning models. The stopwords used for filtering are as follows:

```

1 stopwords = [
2     '\n', 'utf-8', 'False', 'None', 'True', 'and', 'as',
3     'assert', 'async', 'await', 'break', 'class', '
4     continue',

```

```

3      'def', 'del', 'elif', 'else', 'except', 'finally', '
      for', 'from', 'global', 'if', 'import',
4      'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass
      ', 'raise', 'return', 'try', 'while', 'with',
5      'yield', '+', '-', '*', '/', '%', '**', '//', '=', '
      ==', '!=', '<', '>', '<=', '>=', '&', '|',
6      '^', '~', '<<', '>>', '(', ')', '[', ']', '{', '}',
      '.', ',', ':', ';', '"', "'", '\\', '@', '#', '$'
7  ]

```

Listing 5.5: Stopwords Used for Filtering

5.2 Model Selection and Architecture

We used a diverse set of Machine Learning models to approach this problem, promising a balanced comparison across multiple aspects. The selected models can be categorized as follows:

1. Linear Models:

- Logistic Regression is effective for binary classification thanks to its simplicity and interpretability. Logistic Regression has been used in vulnerability detection in source code, more specifically for the binary classification of vulnerabilities in source code. Tang et al. , used Logistic Regression to create an interpretable vulnerability detection model. Their study showed that Logistic Regression could effectively identify vulnerabilities in source code, this served as a basis for assessing the efficacy of other interpretability methods. [20]
Similarly, Munonye and Péter utilized Logistic Regression in the detection of vulnerabilities within OAuth 2.0 workflows. Their study emphasized the algorithm's computational efficiency and interpretability, that made it an effective choice for identifying vulnerabilities in sophisticated authentication and authorization workflows. [28]

2. Tree-Based Models:

- Random Forest and Gradient Boosting Classifier were employed. These models have been applied for vulnerability detection in source code thanks to their ability in handling non-linear relationships and estimating feature importance. Rajapaksha et al. demonstrated that Random Forest performs rather well in binary classification tasks, however, Gradient Boosting outperforms it in multi-class classification [35]. Pakshad et al. utilized these models to identify vulnerable functions by analyzing properties in the structure of code, assisting in targeted security testing [32]. Another study focused on the effectiveness of these classifiers in leveraging Explainable AI for understanding vulnerable code [36].

3. Support Vector Machines (SVM):

- Support Vector Machines (SVMs) have a certain amount of flexibility allowing them to handle both linear and non-linear data distributions utilizing various kernel functions. Linear kernels are suitable for datasets where classes are linearly separable, effectively finding a hyperplane that maximizes the margin between classes. Non-linear kernels, for more complex datasets, such as the Radial Basis Function (RBF) are used to capture nuanced decision boundaries by mapping input features into higher-dimensional spaces. This flexibility allows SVMs to model complex relationships within data, making them effective in various applications, including vulnerability detection in source code. Mouine et al. utilized SVMs with various kernel-based approaches and embedding techniques, such as Bag-of-Words and FastText, to analyze Java source code. Their results demonstrated that SVMs are capable of identifying vulnerabilities effectively, capturing complex patterns in structured datasets, though they were generally outperformed by models such as Random Forest in specific scenarios. [27]

4. Naive Bayes:

- **Multinomial Naive Bayes** has been effectively used in text-related tasks due to its reliance on token frequency distributions.

Anantawijaya and Mauritsius demonstrated its' efficacy in vulnerability detection by analyzing PHP code in the Senayan Library Management System. Using the Bag-of-Words approach for vectorization, their model achieved a remarkable accuracy of 97%, demonstrating the utility of Multinomial Naive Bayes in identifying code vulnerabilities. [4]

5. Ensemble Methods and AutoML:

- **XGBoost**, a gradient boosting-based ensemble method, was utilized for its high performance in tabular data tasks.
- **FLAML (AutoML)** was employed for automated model selection and hyperparameter tuning, ensuring optimal configurations.

Model Type	Description and Key Insights
Linear Models	Logistic Regression: Effective for binary classification with simplicity and interpretability. Applied in detecting vulnerabilities in source code and OAuth 2.0 workflows. Examples: Tang et al. [20], Munonye and Péter [28].
Tree-Based Models	Random Forest & Gradient Boosting: Good at handling non-linear relationships and estimating feature importance. Used in vulnerability detection and explainable AI for code analysis. Examples: Rajapaksha et al. [33], Pakshad et al. [30], Explainable AI study [34].
Support Vector Machines (SVM)	Flexible for both linear and non-linear data using kernel functions (linear and RBF). Suitable for vulnerability detection, analyzing Java code with embeddings. Examples: Mouine et al. [27].
Naive Bayes	Multinomial Naive Bayes: Efficient in text-based tasks using token frequency distributions. Achieved high accuracy in PHP code vulnerability detection. Example: Anantawijaya and Mauritsius [4].
Ensemble Methods	XGBoost: Known for high performance in tabular data tasks.
AutoML	FLAML: Automates model selection and hyperparameter tuning, ensuring optimal configurations.

Table 5.1: Summary of Selected Models and Their Key Features

Chapter 6

Training Process

The training process was designed to systematically prepare, train, and optimize machine learning models for classifying API authentication source code as secure or insecure. It included data splitting, feature representation, model selection, hyperparameter tuning, and finally, evaluation.

6.1 Data Splitting

The dataset was split into training and testing subsets using an 80:20 ratio, ensuring that 80% of the data was used for training the models and 20% was reserved for evaluating model performance [30]. The split ensured the models were tested on unseen data, reducing the risk of overfitting.

```
1 from sklearn.model_selection import train_test_split
2
3 # Splitting the dataset
4 X_train, X_test, y_train, y_test = train_test_split(
5     vectors, labels, test_size=0.2, random_state=42
6 )
```

Listing 6.1: Data Splitting Process

6.2 Feature Representation

Feature representation involved converting raw Python code into numerical representations using two techniques namely, Count Vectorization and

Term Frequency-Inverse Document Frequency (TF-IDF). Count Vectorization counts the frequency of each token in the snippet creating a matrix representation in order to capture term frequencies for a relatively straightforward analysis. These raw frequencies may however not provide enough context for the models. TF-IDF Adjusts aforementioned frequencies by considering their presence across all code snippets, adding an emphasis on unique and relevant terms whilst reducing the weight of common ones. This helps highlight important and unique terms that may have a better impact on the prediction of class labels, such as insecure or secure code. While both methods were utilized in this study, TF-IDF generally provides better feature weighting, which often results in better model performance, particularly when dealing with highly imbalanced data like vulnerabilities in code.

```
1 from sklearn.feature_extraction.text import
   CountVectorizer, TfidfVectorizer
2
3 # Count Vectorization
4 vectorizer = CountVectorizer()
5 count_vectors = vectorizer.fit_transform(X_train)
6
7 # TF-IDF Vectorization
8 tfidf_vectorizer = TfidfVectorizer()
9 tfidf_vectors = tfidf_vectorizer.fit_transform(X_train)
```

Listing 6.2: Vectorization Techniques

6.3 Model Training and Hyperparameter Tuning

Several supervised machine learning models were trained and optimized, including:

- **Logistic Regression:** A linear classifier providing a baseline for performance.
- **Random Forest:** An ensemble model combining multiple decision trees to improve prediction robustness.
- **Gradient Boosting:** A sequential ensemble method that corrects errors from previous iterations.

- **Support Vector Machines (SVM):** A classifier aimed at finding an optimal hyperplane for separating classes.
- **XGBoost:** An advanced boosting algorithm that builds strong models by combining weak learners in a gradient-boosting fashion.
- **Voting Classifier:** An ensemble method that combines predictions from multiple models to improve overall accuracy.
- **FLAML (AutoML):** A framework that automatically selects the best model and tunes hyperparameters to maximize performance.

Hyperparameter tuning was performed using Grid Search and Random Search to identify the optimal configuration for each model. For example, the following code illustrates tuning for the Gradient Boosting model:

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.ensemble import GradientBoostingClassifier
3
4 # Defining parameter grid
5 param_grid = {
6     'n_estimators': [50, 100, 150],
7     'learning_rate': [0.01, 0.1, 0.2],
8     'max_depth': [3, 5, 7]
9 }
10
11 # Grid Search
12 grid_search = GridSearchCV(GradientBoostingClassifier(),
13                             param_grid, cv=3, verbose=2)
14 grid_search.fit(X_train, y_train)
15
16 # Best parameters and model
17 best_model = grid_search.best_estimator_

```

Listing 6.3: Hyperparameter Tuning with Grid Search

6.4 Evaluation Metrics

To assess the effectiveness of the models, we used several performance metrics: **accuracy**, **precision**, **recall**, **F1-score**, and the **confusion matrix**. These metrics provide a comprehensive view of how well

the model performs in terms of both correctly classifying secure and insecure code and minimizing errors. Accuracy measures the overall proportion of correct predictions, while precision focuses on how many of the predicted positive instances were actually positive. Recall, assesses how well the model identifies all true positive instances. The F1-score, balances precision and recall, it is especially useful in cases where the dataset is imbalanced. Finally, the confusion matrix provides a breakdown of true positives, false positives, true negatives, and false negatives, allowing for more detailed error analyses.

```
1 from sklearn.metrics import classification_report,
   accuracy_score, confusion_matrix
2
3 # Making predictions and evaluating
4 y_pred = best_model.predict(X_test)
5 print("Accuracy:", accuracy_score(y_test, y_pred))
6 print("Classification Report:\n", classification_report(
   y_test, y_pred))
7 print("Confusion Matrix:\n", confusion_matrix(y_test,
   y_pred))
```

Listing 6.4: Model Evaluation Example

The models were trained on the training dataset using the vectorized features and evaluated on the test dataset. The training process was repeated multiple times to ensure that the results were consistent and not biased by random initialization or other variations. Once the models were trained, predictions were made on the test set, and the performance was measured using the aforementioned evaluation metrics.

We followed a systematic approach for optimal model performance. Data splitting was used for an unbiased evaluation, feature representation techniques such as TF-IDF and Count Vectorization were used to enable certain models to process the text data. We used an exhaustive process spanning a wide set of hyperparameters to maintain a thorough approach. We finally evaluated the performance based on multiple metrics as well as a confusion matrix to be able to view performance through a multitude of perspectives.

Chapter 7

Model Evaluation and Findings

This section showcases and discusses the performance of our machine learning models. Our evaluation metrics include accuracy, precision, recall, F1-Score as well as Confusion Matrices. The goal of all these matrices is to provide an enhanced understanding of the efficacy as well as performance of aforementioned models.

7.1 Logistic Regression

The logistic regression model demonstrated somewhat limited capability in distinguishing between Error and Good classes achieving a test accuracy of 50%, as seen in the balanced but low precision and recall values. The equal distribution of false positives and false indicate that the model does not favor one class over the other but struggles overall in accurate classification.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.57	0.57	0.57	50%
Good	0.40	0.40	0.40	
Weighted Avg	0.50	0.50	0.50	

Table 7.1: Logistic Regression Performance

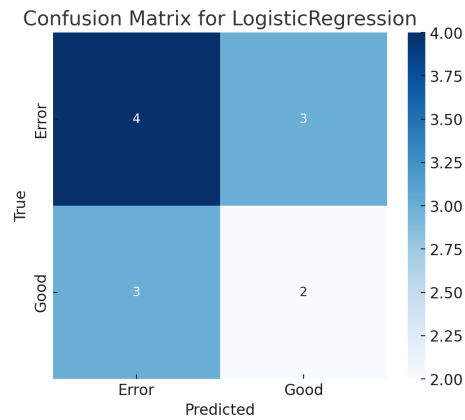


Figure 7.1: Confusion Matrix for Logistic Regression

7.2 Random Forest Classifier

The Random Forest Classifier showed a stronger ability to detect the 'Error' class, shown by its higher recall and F1-Score. The model achieved a test accuracy of 66.7%, However, its performance in identifying the 'Good' class was relatively less robust, with lower recall and F1-Score values. This may be attributed to a class imbalance, in terms of length for our dataset, or simply the model's tendency to favor the 'Error' class.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.67	0.80	0.73	66.7%
Good	0.67	0.40	0.50	
Weighted Avg	0.67	0.67	0.65	

Table 7.2: Random Forest Classifier Performance

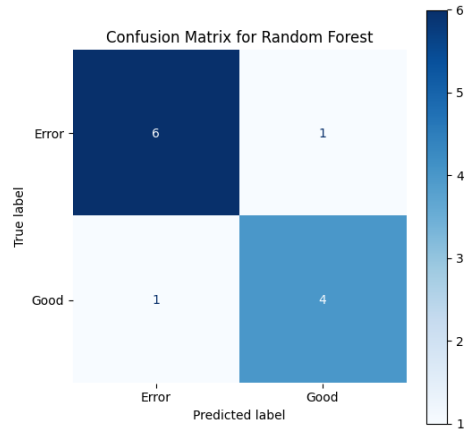


Figure 7.2: Confusion Matrix for Random Forest Classifier

7.3 Support Vector Machine (SVM)

The SVM with radial basis function (RBF) kernel achieved a test accuracy of 66.7% having a somewhat balanced performance between the 'Error' and 'Good' classes. While comparable to Random Forest in terms of accuracy, it demonstrated better precision for the 'Error' class.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.71	0.71	0.71	66.7%
Good	0.60	0.60	0.60	
Weighted Avg	0.67	0.67	0.67	

Table 7.3: SVM Performance

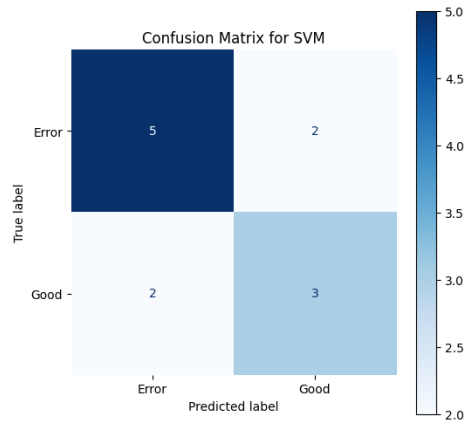


Figure 7.3: Confusion Matrix for SVM

7.4 Gradient Boosting Classifier

The Gradient Boosting Classifier demonstrated robust performance, with a test accuracy of 75% after carefully tuning its hyperparameters. This model was able to effectively capture non-linear dependencies in the data. The precision remained balanced across classes. The disparity in recall highlights areas for improvement, more specifically in detecting "Good" instances of code.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.75	0.86	0.80	75%
Good	0.75	0.60	0.67	
Weighted Avg	0.75	0.75	0.74	

Table 7.4: Gradient Boosting Classifier Performance

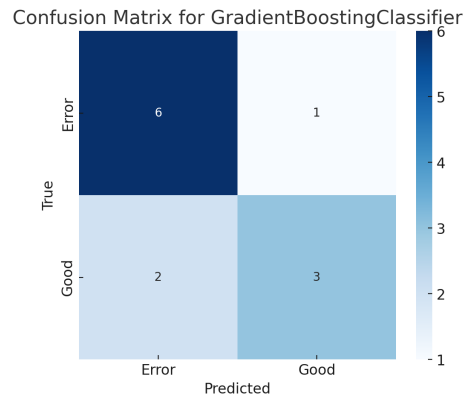


Figure 7.4: Confusion Matrix for Gradient Boosting Classifier

7.5 Extra Trees Classifier

The Extra Trees Classifier emerged as the best-performing model with a test accuracy of 83.3%. It demonstrated strong precision and recall for the "Error" class, with an adequate performance for the "Good" class. Its ability to leverage ensemble techniques and random feature selection contributed significantly to its high performance.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.78	1.00	0.88	83.3%
Good	1.00	0.60	0.75	
Weighted Avg	0.87	0.83	0.82	

Table 7.5: Extra Trees Classifier Performance

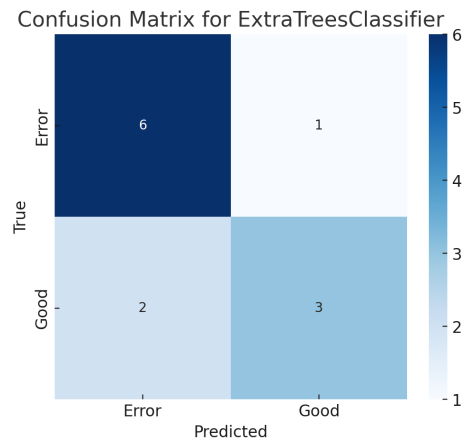


Figure 7.5: Confusion Matrix for Random Forest Classifier

7.6 XGBoost Classifier

XGBoost achieved a test accuracy of 66.7%, showcasing its strong potential when combined with effective hyperparameter tuning. It had a precision of 0.75 for both "Error" and "Good" classes, indicating that 75% of the instances predicted as each class were correct. This suggests the model is equally cautious in predicting both classes, minimizing false positives. Despite its high computational cost, the model performed rather well in precision for the "Good" class.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.75	0.86	0.80	66.7%
Good	0.75	0.60	0.67	
Weighted Avg	0.75	0.75	0.74	

Table 7.6: XGBoost Classifier Performance

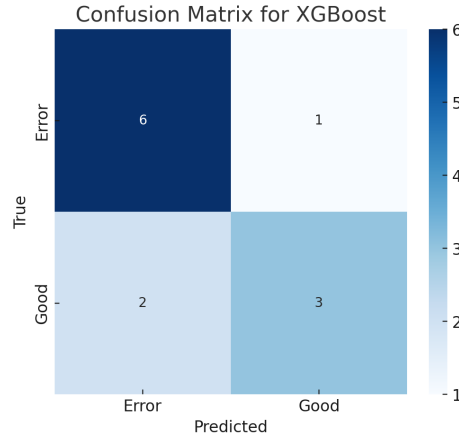


Figure 7.6: Confusion Matrix for XGBoost

7.7 FLAML AutoML

FLAML AutoML verified the Extra Trees Classifier as the best-performing model with a test accuracy of 83.3%. Its automated hyperparameter optimization and model selection capabilities really streamlined our evaluation process. The model, after FLAML, configuration had a precision of 1.00 for the "Good" class and a recall of 1.00 for the 'Error' class. This means that it successfully predicted all "Good" and all "Error" instances accurately.

Metric	Precision	Recall	F1-Score	Accuracy
Error	0.78	1.00	0.88	83.3%
Good	1.00	0.60	0.75	
Weighted Avg	0.87	0.83	0.82	

Table 7.7: FLAML AutoML Performance

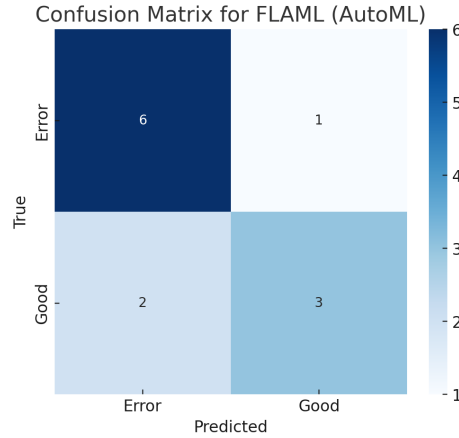


Figure 7.7: Confusion Matrix for FLAML (Extra Trees)

7.8 Summary of Findings

The Extra Trees Classifier and Gradient Boosting emerged as the top performing models, with accuracies of 83.3% and 75%, respectively. Ensemble methods consistently outperformed linear classifiers, highlighting the importance of capturing complex feature interactions. Future work could explore hybrid models and advanced embeddings to further improve performance.

Model	Precision	Recall	F1-Score	Accuracy
ExtraTrees	87.0%	83.0%	82.0%	83.3%
FLAML(ET)	87.0%	83.0%	82.0%	83.3%
Gradient Boosting	75.0%	75.0%	74.0%	75.0%
Linear Regression	50.0%	50.0%	50.0%	50.0%
Random Forest	67.0%	67.0%	65.0%	66.7%
Support Vector Machine	67.0%	67.0%	67.0%	66.7%
Extreme Gradient Boosting	75.0%	75.0%	74.0%	66.7%

Table 7.8: Performance Metrics for All Models

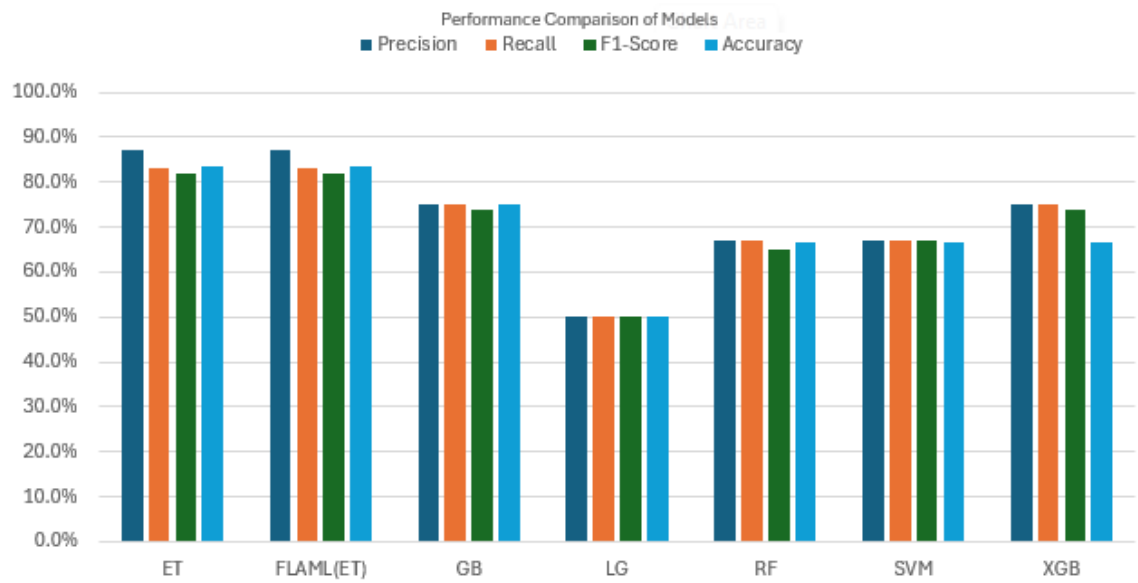


Figure 7.8: Performance Comparison of Models

Chapter 8

Explainable artificial intelligence (XAI)

This section details the integration of Explainable AI (XAI) techniques into the machine learning pipeline to enhance interpretability and trust in model predictions. The focus is on ensuring that the Extra Trees Classifier, identified as the most effective model, provides transparent decision-making insights for developers and stakeholders.

8.1 Motivation for XAI

Explainable AI bridges the gap between high-performing machine learning models and human understanding. In the context of API vulnerability scanning, XAI helps identify key features contributing to security classifications, making the models more actionable for developers [31, 40]. By integrating XAI, we aim to:

- Identify critical features that influence the classification of secure and insecure code.
- Provide insights into model decision boundaries for improved debugging and optimization.
- Enhance stakeholder trust by demonstrating model accountability.

8.2 SHAP for Feature Importance Analysis

SHapley Additive exPlanations (SHAP) was employed to quantify feature importance and visualize model interpretability. SHAP values provide additive feature attributions, indicating the contribution of each token to the prediction outcome. This technique is particularly suited for ensemble methods like Extra Trees Classifier.

Implementation Details

1. **Preprocessing:** Code snippets were preprocessed to remove comments, filter out stopwords, and tokenize the content. These steps were based on the Dataset preparation activities during Model Training stage. This is important as the dataset were transformed into numerical values. For interpretability we need to convert it back into keywords.
2. **Model Loading:** The Extra Trees Classifier model was loaded using the `pickle` library, and predictions were generated for a subset of the dataset.
3. **SHAP Integration:** The SHAP library was used to calculate feature contributions for individual predictions. The `TreeExplainer` module was utilized to interpret the ensemble model.

```
1 import shap
2 import pickle
3
4 # Load the trained model
5 with open('extra_tree_classifier.pkl', 'rb') as f:
6     model = pickle.load(f)
7
8 # Initialize SHAP TreeExplainer
9 explainer = shap.TreeExplainer(model)
10
11 # Compute SHAP values
12 shap_values = explainer.shap_values(X_test)
13 shap.summary_plot(shap_values, X_test, feature_names=
    feature_names)
```

Feature Importance Results

Figure 8.1 presents the SHAP summary plot, highlighting the most influential features. Tokens such as ‘`authorization`’, ‘`admin`’, and ‘`token`’ were identified as key indicators for distinguishing between secure and insecure API code. These findings align with domain knowledge, emphasizing the role of authentication mechanisms in API vulnerabilities.

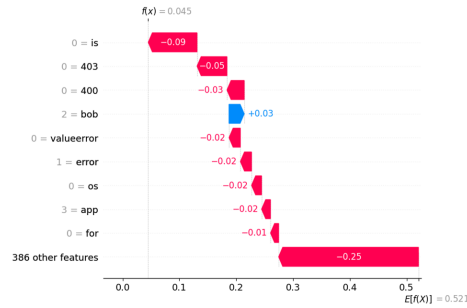


Figure 8.1: SHAP Summary Plot for Feature Importance

8.3 Impact of XAI on Model Optimization

The SHAP waterfall diagram provides specific insights for the individual contributions of source code features for a vulnerability prediction. In this example, the predicted output is represented as $E[f(x)] = 0.521$, while the actual contribution of features yields $f(x) = 0.045$. This difference underscores the importance of source code feature-level explainability in trying to interpret model behavior.

From the diagram:

- Missing features such as ‘`is`’ (-0.09), ‘`403`’ (-0.05), and ‘`400`’ (-0.03) had significant negative impacts on the prediction.
- The feature ‘`bob`’ contributed positively ($+0.03$) to the prediction.

- Other features, including ‘valueerror’ (-0.02), ‘error’ (-0.02), and ‘os’ (-0.02), showed minor negative impacts.
- A large number of features (386 in total) collectively contributed to a decrease of -0.25 in the prediction value, indicating the cumulative effect of low-impact features.

8.4 Discussion and Conclusion

The integration of SHAP-based explainable AI (XAI) techniques into the model evaluation process has proven invaluable. The transparent visualization of feature contributions, as seen in the waterfall diagram, facilitates better decision-making in model refinement. For instance, excluding tokens with low SHAP contributions from the input space reduces noise and enhances generalizability. Furthermore they provide insights into how these models are making their predictions and which aspects of the source code are impacting these predictions.

Furthermore, emphasizing important tokens (e.g., those related to security concepts like ‘authorization’ and ‘token’) ensures that the model prioritizes key aspects of the input data. This approach not only improves interpretability but also builds trust in AI systems by offering a clear understanding of the factors influencing predictions. Consequently, these advancements support the development of robust API vulnerability detection frameworks and other critical AI applications.

Chapter 9

Discussion

This study aimed to integrate **Explainable AI (XAI)** techniques into machine learning models for the detection of vulnerabilities in API authentication code. The results of our experiments, involving various machine learning models like Random Forest, Extra Trees, Support Vector Machines (SVM), and XGBoost, highlight the importance of combining both **model performance** and **interpretability** in real-world applications like security.

9.1 Model Performance Evaluation

The models tested in this study demonstrated varying degrees of success, with the **Extra Trees Classifier** performing the best, achieving an accuracy of 83.3%. This model leveraged the power of ensemble learning and benefited from fine-tuning, particularly when compared to simpler models like Logistic Regression and Naive Bayes, which showed relatively lower performance (e.g., Logistic Regression achieved only 50% accuracy).

While tree-based models, including **Random Forest** and **Gradient Boosting**, were more robust in handling the feature interactions present in the dataset, other models like **SVM** and **XGBoost** also showed promising results. These models exhibited high precision, particularly in the “Error” class, making them quite useful for such specific types of vulnerability detection.

A key takeaway from this experiment is that ensemble methods tend to outperform single classifiers thanks to their ability to capture complex interactions between code features, which is particularly valuable in security-

related tasks where vulnerabilities are often nuanced and context-dependent.

9.1.1 Comparing Findings with existing literature

Farasat and Posegga [?] conducted an extensive evaluation of machine learning algorithms for detecting vulnerabilities in Python source code. Their study applied various models, including a Bidirectional Long Short-Term Memory (BiLSTM) network, which achieved superior performance with an average accuracy of 98.6% and an F1-score of 94.7%. This high efficacy is attributed to the model’s ability to capture contextual information within code sequences, enhancing the detection of complex vulnerabilities. Their research underscores the potential of deep learning architectures in improving vulnerability detection systems.

9.2 Impact of Explainable AI

Incorporating **SHAP** for **feature importance** analysis provided a window into how these models make predictions. By leveraging **XAI techniques**, we were able to identify critical features, source code terms, that significantly influenced model decisions. This is essential, as understanding which features are most impactful can help security teams refine API code, highlighting areas that are most prone to vulnerabilities. This can then further be utilized in coding policies and vulnerability prevention.

Despite the success of XAI, we acknowledge the computational complexity introduced by these techniques. While **SHAP** offers precise and detailed insights, its computational cost may increase with the size and complexity of the dataset. This trade off must be considered when deploying XAI methods in real-time applications, especially in production environments where speed is critical and resources are minimal.

Chapter 10

Limitations and Future Work

While the study demonstrates promising results, several limitations should be addressed in future work:

- **Dataset Limitation:** The dataset used in this study contained only 60 labeled code snippets. Although it was sufficient for initial exploration, a larger and more diverse dataset would be necessary to validate the results and improve model robustness.
- **Model Variability:** While ensemble models like **Extra Trees** performed well, other models like **XGBoost** and **SVM** showed inconsistent performance. Further tuning and experimentation with different hyperparameter configurations might help enhance their stability and predictive power.
- **Scalability of XAI Techniques:** As the dataset grows, the **computational burden** of XAI methods like **SHAP** may increase. Exploring other model-agnostic interpretability techniques, such as **LIME**, or improving the scalability of **SHAP** would be valuable.
- **Hybrid Models:** Future work could explore combining the strengths of both **unsupervised** and **supervised** learning to create hybrid models that can effectively detect vulnerabilities while maintaining interpretability.

Chapter 11

Conclusion

This research demonstrates the potential of **Explainable AI (XAI)** techniques in enhancing the interpretability and trustworthiness of machine learning models used for **API vulnerability detection**. By integrating **SHAP**, we were able to uncover valuable insights into how the models identify critical features that influence the classification of secure and insecure API authentication code.

Our findings indicate that **ensemble models** like **Extra Trees Classifier** and **Random Forest** are particularly well-suited for vulnerability detection, as they can capture complex feature interactions more effectively than simpler models. However, while **XAI** techniques such as **SHAP** provide transparency, they also introduce **computational challenges** that need to be addressed for large-scale deployment.

This study highlights the importance of interpretability in security-related AI applications, providing developers with actionable insights into the features driving vulnerability predictions. By combining high-performing models with transparent, explainable techniques, we can not only improve model accuracy but also ensure that security decisions are well-understood, trusted, and actionable.

In summary, the integration of **XAI** methods enhances **AI-driven security solutions** by fostering trust, providing transparency, and ultimately guiding developers in building more secure systems. As AI models become more prevalent in critical applications like security, the need for **explainable** and **interpretable** solutions will continue to grow, indicating significant potential for future research and development.

References

- [1] About the OWASP Foundation | OWASP Foundation.
- [2] Akamai: API: The attack surface that connects us all. *Computer Fraud & Security*, 2021(11):4, November 2021.
- [3] Moutaz Alazab, Mamoun Alazab, Andrii Shalaginov, Abdelwadood Mesleh, and Albara Awajan. Intelligent mobile malware detection using permission requests and API calls. *Future Generation Computer Systems*, 107:509–521, June 2020.
- [4] I Komang Sughosa Anantawijaya and Tuga Mauritsius. Detecting vulnerabilities in senayan library management system using machine learning. *Journal of Logistics, Informatics and Service Science*, 11(6):266–278, 2024.
- [5] Sanjay Bahl and O. P. Wali. Perceived significance of information security governance to predict the information security service quality in software service industry: An empirical analysis. *Information Management & Computer Security*, 22(1):2–23, March 2014. Publisher: Emerald Group Publishing Limited.
- [6] H. K. Brar and P. J. Singh. Comparing detection ratio of three static analysis tools. *International Journal of Computer Applications*, 124(13):1–5, 2015.
- [7] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Zhiqiang Lin, and Yinqian Zhang. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *RAID '23 Proceedings*, 2023.
- [8] Zhen Chen, Maosheng Pan, Pengfei He, Wenchao Qi, Linlin Liu, Limin Shen, and Dianlong You. Context and auto-interaction are all you need:

Towards context embedding based QoS prediction via automatic feature interaction for high quality cloud API delivery. *Future Generation Computer Systems*, 128:265–281, March 2022.

- [9] Mounika Deverashetti, Ranjitha K., and Pradeepthi K.v. Security analysis of menstruation cycle tracking applications using static, dynamic and machine learning techniques. *Journal of Information Security and Applications*, 67:103171, June 2022.
- [10] F5 Networks, Inc. Continuous API Sprawl, November 2021. <https://www.f5.com/pdf/reports/f5-office-of-the-cto-report-continuous-api-sprawl.pdf>.
- [11] B. Kumar; O. A. Falhi. Digital Transformation through APIs. In *2022 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COM-IT-CON)*, volume 1, pages 623–628, 2022. Journal Abbreviation: IEEE.
- [12] F. Hussain; W. Li; B. Noye; S. Sharieh; A. Ferworn. Intelligent Service Mesh Framework for API Security and Management. In *2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 0735–0742, 2019. ISSN: 2644-3163 Journal Abbreviation: IEEE.
- [13] OWASP Foundation. Owasp api security top ten, 2023. Accessed: 2024-12-24.
- [14] Gartner. Predicts 2022: Apis demand improved security and management, December 2021. Accessed: 2024-11-10.
- [15] Danielle Gonzalez, Paola Peralta Perez, and Mehdi Mirakhorli. Barriers to shift-left security: The unique pain points of writing automated tests involving security controls. In *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ESEM '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [16] Chang Hoong Jack, See Kwee Teck, Lim Tong Ming, and Ding Ying Hong. An Overview Analysis of Authentication Mechanism in Microservices-Based Software Architecture: A Discussion Paper. In

2023 IEEE 8th International Conference On Software Engineering and Computer Systems (ICSECS), pages 1–6, August 2023.

- [17] Shubhra Kanti Karmaker (“Santu”), Md. Mahadi Hassan, Micah J. Smith, Lei Xu, Chengxiang Zhai, and Kalyan Veeramachaneni. AutoML to Date and Beyond: Challenges and Opportunities. *ACM Comput. Surv.*, 54(8):175:1–175:36, October 2021.
- [18] A. Ramadhan Ghilen; A. Mohamed Zahou; W. Abedelmonem Ben Khalifa. Quantum cryptography for the benefit of API keys safety. In *2022 IEEE 9th International Conference on Sciences of Electronics, Technologies of Information and Telecommunications (SETIT)*, pages 343–348, 2022. Journal Abbreviation: IEEE.
- [19] Wahab Khan, Ali Daud, Jamal A. Nasir, and Tehmina Amjad. A survey on the state-of-the-art machine learning models in the context of NLP. *Kuwait Journal of Science*, 43(4), November 2016. Number: 4.
- [20] Ashish Kumar, Rachit Agarwal, Srinivas R., and et al. Interpretation of learning-based automatic source code vulnerability detection model using lime. *IEEE Transactions on Software Engineering*, 47(12):2412–2426, 2021.
- [21] Cemil Kuzey, Ali Uyar, and Dursun Delen. An investigation of the factors influencing cost system functionality using decision trees, support vector machines and logistic regression. *International Journal of Accounting & Information Management*, 27(1):27–55, March 2019. Publisher: Emerald Publishing Limited.
- [22] Zhen Li, Deqing Zou, and Shouhuai Xu. Vuldeepecker: An automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 2018 ACM Conference*, pages 200–210, 2018.
- [23] Ruitong Liu, Yanbin Wang, Haitao Xu, Bin Liu, Jianguo Sun, Zhenhao Guo, and Wenrui Ma. Source code vulnerability detection: Combining code language models and code property graphs. *arXiv preprint arXiv:2404.14719*, 2023.
- [24] Tina Marjanov, Ivan Pashchenko, and Fabio Massacci. Machine learning for source code vulnerability detection: What works and what isn’t there yet. *IEEE Security & Privacy*, 20(5):60–69, 2022.

- [25] Nuno Mateus-Coelho, Manuela Cruz-Cunha, and Luis Gonzaga Ferreira. Security in Microservices Architectures. *Procedia Computer Science*, 181:1225–1236, January 2021.
- [26] Md Raihan Mia, Hossain Shahriar, Maria Valero, Nazmus Sakib, Bilash Saha, Md Abdul Barek, Md Jobair Hossain Faruk, Ben Goodman, Rumi Ahmed Khan, and Sheikh Iqbal Ahamed. A comparative study on HIPAA technical safeguards assessment of android mHealth applications. *Smart Health*, 26:100349, December 2022.
- [27] Esma Mouine, Yan Liu, Lu Xiao, Rick Kazman, and Xiao Wang. Explaining the contributing factors for vulnerability detection in machine learning. *arXiv preprint arXiv:2406.03577*, 2024.
- [28] Kindson Munonye and Martinek Péter. Machine learning approach to vulnerability detection in oauth 2.0 authentication and authorization flow. *International Journal of Information Security*, 21(2):223–237, 2022.
- [29] Henry R. Neave. Deming’s 14 points for management: Framework for success. *Journal of the Royal Statistical Society. Series D (The Statistician)*, 36(5):561–570, 1987.
- [30] Quang Hung Nguyen, Hai-Bang Ly, Lanh Si Ho, Nadhir Al-Ansari, Hiep Van Le, Van Quan Tran, Indra Prakash, and Binh Thai Pham. Influence of Data Splitting on Performance of Machine Learning Models in Prediction of Shear Strength of Soil. *Mathematical Problems in Engineering*, 2021(1):4832864, 2021. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2021/4832864>.
- [31] Aviv Notovich, Hila Chalutz-Ben Gal, and Irad Ben-Gal. Explainable Artificial Intelligence (XAI): Motivation, Terminology, and Taxonomy. In Lior Rokach, Oded Maimon, and Erez Shmueli, editors, *Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook*, pages 971–985. Springer International Publishing, Cham, 2023.
- [32] Puya Pakshad, Alireza Shameli-Sendi, and Behzad Khalaji Emamzadeh Abbasi. A security vulnerability predictor based on source code metrics. *International Journal of Information Security*, 21:315–327, 2023.

- [33] Lianglu Pan, Shaanan Cohney, Toby Murray, and Van-Thuan Pham. Detecting Excessive Data Exposures in Web Server Responses with Metamorphic Fuzzing. *ARXIV*, January 2023. arXiv:2301.09258 [cs].
- [34] Sampath Rajapaksha, Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. Ai-powered vulnerability detection for secure source code development. In *Innovative Security Solutions for Information Technology and Communications (SecITC 2022)*, volume 13809 of *Lecture Notes in Computer Science*, pages 275–288. Springer, 2023.
- [35] Sampath Rajapaksha, Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. Ai-powered vulnerability detection for secure source code development. In *Proceedings of the International Conference on Software Engineering*, volume 32, pages 251–260, 2023.
- [36] Sampath Rajapaksha, Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. Enhancing security assurance in software development: Ai-based vulnerable code detection with static analysis. In *Proceedings of the Advances in Software Engineering Conference*, pages 310–320, 2023.
- [37] Rebecca L. Russell, Louis Kim, Lei H. Hamilton, Tomo Lazovich, Jacob A. Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. Automated vulnerability detection in source code using deep representation learning. In *17th IEEE International Conference on Machine Learning and Applications (IEEE ICMLA 2018)*, pages 1–8, Orlando, Florida, USA, 2018.
- [38] Waleed Bin Shahid, Baber Aslam, Haider Abbas, Hammad Afzal, and Saad Bin Khalid. A deep learning assisted personalized deception system for countering web application attacks. *Journal of Information Security and Applications*, 67:103169, June 2022.
- [39] Md. Shazzad Hossain Shaon, Alfredo Cuzzocrea, Mst Shapna Akter, and Shahriyar Zaman Ridoy. Enstack: An ensemble stacking framework of large language models for enhanced vulnerability detection in source code. *arXiv preprint arXiv:2411.16561*, 2024.
- [40] B. P. Sheela and H. Girisha. An Exploration on Explainable AI with Background and Motivation for XAI. In Syed Muzamil Basha, Hamed

- Taherdoost, and Cleber Zanchettin, editors, *Innovations in Cybersecurity and Data Science*, pages 481–489, Singapore, 2024. Springer Nature.
- [41] Amit Sheps. Why Shift Right Testing & Security? Shift Right vs. Shift Left.
 - [42] Marco Squarcina, Stefano Calzavara, and Matteo Maffei. The Remote on the Local: Exacerbating Web Attacks Via Service Workers Caches. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 432–443, May 2021.
 - [43] Srinivas Aditya Vaddadi, Ramya Thatikonda, Adithya Padthe, and Pandu Ranga Rao Arnepalli. Shift left testing paradigm process implementation for quality of software based on fuzzy. *Soft Comput*, July 2023.
 - [44] Anthony Viriya and Yohan Muliono. Peeking and Testing Broken Object Level Authorization Vulnerability onto E-Commerce and E-Banking Mobile Applications. *Procedia Computer Science*, 179:962–965, January 2021.
 - [45] Alexey Vishnyakov, Daniil Kuts, Vlada Logunova, Darya Parygina, Eli Kobrin, Georgy Savidov, and Andrey Fedotov. Sydr-fuzz: Continuous hybrid fuzzing and dynamic analysis for security development lifecycle. In *2022 Ivannikov Ispras Open Conference (ISPRAS)*, pages 111–123, 2022.
 - [46] Mohammad Wazid, Ashok Kumar Das, Vinay Chamola, and Youngho Park. Uniting cyber security and machine learning: Advantages, challenges and future research. *ICT Express*, 8(3):313–321, September 2022.
 - [47] David A. Wheeler. Flawfinder. <https://dwheeler.com/flawfinder/>. Accessed: 2024-12-11.
 - [48] Chamila Wijayarathna and Nalin Asanka Gamagedara Arachchilage. Using cognitive dimensions to evaluate the usability of security APIs: An empirical investigation. *Information and Software Technology*, 115:5–19, November 2019.
 - [49] Fiorella Zampetti, Simone Scalabrino, Rocco Oliveto, Gerardo Canfora, and Massimiliano Di Penta. How open source projects use static code

analysis tools in continuous integration pipelines. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pages 334–344, 2017.